

VECoDI: Verifiable and Confidential DNN Inference on Low-End Edge Devices

Anonymous Author(s)
Anonymous Institution
Anonymous City, Anonymous Country

Abstract

Deploying deep neural network (DNN) inference on low-end edge devices raises two key challenges: protecting model confidentiality against a potentially compromised edge system and enabling verifiable inference without incurring prohibitive overhead. Existing approaches either house (partial) models and inference software within trusted execution environments (TEEs), resulting in high cost and an application-dependent trusted computing base (TCB), or execute in untrusted environments, providing little security.

In this work, we present VECODI, a framework for verifiable and confidential DNN inference on constrained edge devices. At its core, VECODI introduces SHANGRI-LA, a new execution abstraction on TrustZone-M TEEs that establishes a third runtime environment with privileges strictly between the Secure and Non-Secure Worlds. VECODI leverages SHANGRI-LA to execute untrusted inference code in the Non-Secure World while leveraging minimal application-agnostic Secure-World support to protect model confidentiality and enable verifiability (with respect to proper execution of inference code and model parameters) of inference results.

We realize VECODI on a real-world NUCLEO-L552ZE-Q development board. Our results show VECODI's small TCB, memory footprint, and runtime overhead, making it a practical option for securing inference procedures in low-end edge devices.

CCS Concepts

• **Security and privacy** → **Embedded systems security**; • **Computing methodologies** → *Neural networks*; • **Computer systems organization** → *Embedded systems*.

Keywords

Trusted execution, embedded machine learning, model confidentiality, proof of execution, TrustZone-M

ACM Reference Format:

Anonymous Author(s). 2026. VECODI: Verifiable and Confidential DNN Inference on Low-End Edge Devices. In . ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Edge computing has emerged as a paradigm for performing inference of machine learning (ML) models, particularly deep neural networks (DNNs), in latency-sensitive and resource-constrained environments [20]. In this paradigm, DNN models are trained in centralized infrastructures (e.g., the cloud) by service providers, referred to as model providers, and subsequently deployed to edge devices, such as sensor and actuator microcontrollers. These low-cost devices then perform DNN inference locally, enabling real-time

decision-making for several modern IoT applications such as remote medical diagnostics, autonomous systems, and industrial monitoring [20]. However, moving from centralized to decentralized deployments creates new attack vectors.

First, training DNN models typically requires substantial computational resources, as well as significant effort in collecting and curating training data. As such, model providers often consider these models to be valuable intellectual property/assets that must be protected [45]. However, when deployed at the edge, DNN models are exposed to stronger adversarial capabilities, where device-controlled adversaries have both physical and software access to the device. This enables such adversaries to: (1) directly extract the model, either at rest (e.g., from on-device flash memory) or at runtime, and (2) mount *model extraction* attacks [39] by reconstructing the deployed model via performing a large number of carefully crafted inference queries. These threats introduce a new challenge of *model confidentiality* for model providers in edge deployments.

Second, edge inference outcomes are often used to drive critical decision-making. For example, in autonomous systems, object detection results are used to identify obstacles (e.g., pedestrians) and trigger braking upon detection. Similarly, in remote medical diagnostics, ECG analysis results may be used to detect cardiac abnormalities and initiate timely intervention [20]. However, edge devices operate under strict constraints in terms of cost, size, and power, which limit the deployment of sophisticated security mechanisms. As a result, these low-end devices are more vulnerable to malware infestation, leading to a surge in IoT malware targeting edge devices in recent years [33]. Once compromised, such malware can take control of the entire software stack and produce spoofed inference results to manipulate downstream decisions, potentially endangering physical environments and users. To address this second class of attacks, *verifiable inference* is necessary. In traditional centralized settings, this is typically not a major concern, as the model providers have full control over powerful inference devices.

To enable *model confidentiality* with low performance overhead, recent work leverages Trusted Execution Environments (TEEs), giving rise to a class of techniques known as *TEE-Shielded DNN Partitioning (TSDP)* [15, 19, 25, 36]. These techniques partition a DNN into two disjoint components: (i) *privacy-sensitive* and (ii) *privacy-insensitive* parts. The privacy-sensitive components are executed within the TEE, while the remaining parts are left unprotected. Despite good performance, this line of work suffers from a key limitation: it enlarges the trusted computing base (TCB) by incorporating DNN inference code directly into the TEE, thereby increasing both the software footprint and the attack surface to already-constrained/vulnerable edge devices. Moreover, many of

these approaches [15, 25, 36] overlook model extraction attacks, leaving this important issue unaddressed.

On the other hand, prior work on *verifiable inference* either assumes that the entire inference pipeline is executed within TEEs [4, 10, 43] or relies on cryptographic techniques in conjunction with TEEs [38, 47]. The former is often undesirable in edge settings, as DNN models may exceed the limited memory capacity of the TEE. The latter, while more flexible, can introduce significant computational overhead, making it unsuitable for power-constrained devices. More importantly, both approaches still suffer from TCB bloat due to the inclusion of complex and potentially buggy inference code within the TEE. A compromise of such code could escalate into other high-assurance components (e.g., secure key storage, secure software update) that live in the same TEE, as is the case with ARM TrustZone-M – a commonly used TEE for low-end edge devices.

In light of these challenges, we introduce VECODI, a TrustZone-M-based framework that simultaneously enables *model confidentiality* and *verifiable inference* for DNNs on low-end edge devices. Unlike prior TEE-based approaches, VECODI does not place DNN inference code or data in the Secure World. Instead, it introduces a new abstraction, SHANGRI-LA, which serves as a third runtime environment alongside TrustZone-M's Secure and Non-Secure Worlds.

SHANGRI-LA resides in the Normal World memory but operates with elevated privileges enforced by the Secure World. This design enables reuse of existing Normal World functionality, avoiding cross-world code duplication. At the same time, SHANGRI-LA guarantees the confidentiality and integrity of its code and data—both at rest and during execution—even when the Normal World is untrusted. In addition, SHANGRI-LA can directly access hardware I/O peripherals, preventing interference from the Normal World on the physical acquisition of data. Despite these capabilities, it remains less privileged than the Secure World, ensuring that a compromised or faulty SHANGRI-LA cannot impact Secure World components. Finally, SHANGRI-LA requires only minimal, application-agnostic code in the Secure World. This code can be thoroughly tested or formally verified once and reused across multiple SHANGRI-LA applications.

At the protocol level, we combine SHANGRI-LA isolation abstraction with two key features. First, an inference authorization protocol allows the model provider to limit the number/frequency of inferences that can be performed; this mitigates *model extraction attacks* that threaten *model confidentiality*. Second, we extend SHANGRI-LA to generate a verifiable proof of inference execution that accompanies each inference result. This proof allows an external verifier to assess whether: (1) the inference inputs were correctly captured from physical interfaces; (2) inference code is executed correctly on the intended DNN model and authentic input; and (3) the reported inference result corresponds exactly to this execution instance. This also supports privacy, since the proof validation does not require raw inputs used by the edge device that may contain sensitive user information (e.g., consider an image recognition case).

Contributions. Our main intended contributions are as follows:

1- Third Runtime Environment. We introduce SHANGRI-LA, a lightweight execution abstraction on top of TrustZone-M. It provides a third runtime environment that is protected from Normal

World compromise, while ensuring that any compromise within SHANGRI-LA cannot affect the Secure World. At the same time, SHANGRI-LA physically lives in Normal World and thus can reuse existing Normal World code, reducing the overall software footprint. We believe SHANGRI-LA is of independent interest beyond secure DNN inference, as it enables third-party developers to deploy arbitrary trusted applications on TrustZone-M without modifying core Secure World components.

2- Secure and Verifiable DNN Inference. We build upon SHANGRI-LA abstraction to achieve both *model confidentiality* and *verifiable inference* on low-end edge devices. First, we equip SHANGRI-LA with an inference authorization mechanism to mitigate model extraction attacks. Second, we enable SHANGRI-LA to generate verifiable proofs of input retrieval and inference execution, allowing external verifiers to validate results without accessing sensitive raw input data. Along with these extensions, SHANGRI-LA is used to construct VECODI: a protocol executed between inference stakeholders to realize confidential and verifiable DNN inference.

3- Practical Deployment and Evaluation. We implement VECODI on real-world hardware, particularly prototyping and evaluating an VECODI-enabled device (DEV) on a NUCLEO-L552ZE-Q development board [35]. Our evaluation results demonstrate the practical benefits of VECODI: Compared to prior TEE-based solutions, VECODI eliminates the need to place inference code in the Secure World, reducing the TCB and overall software footprint by $x\%$ and $y\%$, respectively. Moreover, VECODI incurs only negligible runtime overhead, making it well-suited for resource-constrained devices. Finally, we demonstrate real-world applicability through a case study on camera-based edge image classification.

2 Background

2.1 TrustZone for ARM Cortex-M MCUs

ARM TrustZone-M provides hardware-enforced isolation for Cortex-M microcontrollers by partitioning the system into two domains: the Secure World and the Normal World. Each domain can only access memory regions assigned to its corresponding security state.

Memory regions, including code, data, and peripherals, are classified as Secure, Non-Secure, or Non-Secure Callable (NSC) through the Security Attribution Unit (SAU) and the Implementation-Defined Attribution Unit (IDAU), both configured by Secure-World code at boot time. This ensures that only Secure-World code can access Secure memory, while Non-Secure code is restricted to Non-Secure regions.

To enable controlled invocation, the Secure World exposes APIs that can be called by the Normal World through designated NSC entry points. These entry points are the only locations where control can transfer from the Non-Secure World to the Secure World, preventing arbitrary jumps into Secure-World code. As a result, the Normal World cannot tamper with the execution of Secure-World code except through explicitly defined APIs, and cannot access Secure-World resources protected by the SAU and IDAU, ensuring strong isolation between these two worlds.

In practice, the Secure World contains a *small* trusted computing base (TCB) consisting of security-critical components (e.g., secure boot, key storage, and attestation) to minimize the attack surface. In

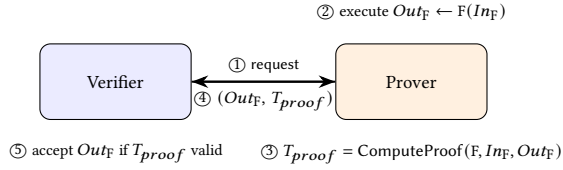


Figure 1: Generic Proof-of-Execution (PoX) workflow.

contrast, the Non-Secure World hosts a significantly larger software stack, including the operating system and user applications.

We also note that, unlike the IDAU, the SAU can be reconfigured at runtime by the Secure World, allowing memory regions to transition dynamically between Secure and Non-Secure states.

2.2 Remote Attestation and Proof of Execution

Remote Attestation (RA). RA allows a remote verifier to establish trust in the software state of a target device [23]. To be secure, RA requires a root of trust (RoT) on the target device (e.g., TrustZone-M TEE) to measure the software to be attested and sign (or MAC) the measurement result under a device-specific attestation key, yielding attestation evidence. The verifier can validate the evidence by checking the signature/MAC using the pre-shared verification key and matching the measurement against a known-good (golden) reference. While RA certifies *what* software is installed, it offers no guarantee over *what the device subsequently computes*.

Proof of Execution (PoX). PoX extends RA evidence to include execution information [30]. The RoT attests not only to code identity but to the fact that a designated function F ran to completion on a given input and produced a given output within the attested device. As illustrated in Figure 1, the protocol follows a challenge-response exchange between a verifier and a prover device:

1- Request. Verifier issues request containing a cryptographic challenge. It also specifies which function F should be executed and optionally a verifier-provided input In_F .

2- Execution. Upon receiving it, the prover computes $Out_F \leftarrow F(In_F)$; the RoT monitors this execution and immediately aborts it if the execution is tampered with. Note that F code may contain direct access to hardware I/O peripherals for local acquisition of data (e.g., when performing inference on an edge sensor device).

3- Proof generation. The RoT produces attestation evidence T_{proof} . This token should be bound to the challenge received in the request, the executed F binary, its legal entry point, In_F , and Out_F . Crucially, T_{proof} should only be produced if F execution is completed successfully and (for timeliness) after the receipt of the challenge it is bound to.

4- Response. The prover returns (Out_F, T_{proof}) ; the verifier accepts Out_F as authentic if and only if T_{proof} is successfully verified using the verification key and the measurement matches the expected challenge, F binary, received Out_F , and expected input In_F (if any).

2.3 TEE-Shielded DNN Partitioning

Running a full DNN inside a low-end TrustZone-M TEE is generally infeasible: Secure RAM is measured in tens of kilobytes, whereas even compact models require orders of magnitude more memory.

TEE-Shielded DNN Partitioning (TSDP) addresses this by decomposing a model M into two sub-networks [15, 19, 25]: a sensitive component M_{pri} sequestered in the Secure World, and a public component M_{pub} executed in the Normal World under the device’s full memory and compute budget. Activations cross the security boundary at the partition point; each crossing incurs a context-switch penalty whose cost grows with the number of inter-domain transitions.

Partition objectives. The choice of boundary is dictated by the target security property:

IP protection. TeeSlice [19] and SecureQNN [5] isolate bottleneck layers whose parameters are critical to model utility, raising the cost of surrogate-model reconstruction via extraction or distillation attacks [44].

Input privacy. ShadowNet [36] and SeITZ [16] shield the initial layers, which are most susceptible to feature-inversion and membership-inference attacks against raw input data.

Execution efficiency. SmartZone [17] minimizes world-switch frequency by colocating layers that exchange large activation tensors, bounding latency to the number of partition crossings rather than model depth.

Regardless of objective, TSDP requires hosting M_{pri} alongside a full inference engine (e.g., TFLite Micro) inside the Secure World, inflating the TCB beyond the minimality required for a secure RoT. End-to-end latency is similarly constrained: on cycle-limited MCUs, fine-grained partitions that maximize security coverage do so at the cost of frequent pipeline stalls on each domain crossing.

3 System/Threat Models and Goals

3.1 System Model

We consider an edge inference setting that comprises four stakeholders:

Model provider (PVD): curates training data, provides the DNN model M , and authorizes access to it.

Edge device (DEV): edge device running on single-core MCU equipped with a low-cost TrustZone-M TEE. Note that MCUs lack memory management units, making them unsuitable for isolation mechanisms based on virtual memory (e.g., operating system- or hypervisor-based). Our model reflect this.

Model consumer (CSM): obtains authorization from PVD to run inference on DEV.

Verifier (VRF): receives inference results and validates their trustworthiness before downstream use. Although logically distinct, VRF’s role can be implemented by another entity, e.g., in practice, a single entity can play both VRF and CSM roles, where the consumer of a result also verifies locally verifies it.

As illustrated in Figure 2, secure edge inference consists of four interactions among these stakeholders:

I1: Provisioning. PVD securely provisions DEV with required cryptographic material and installs the input-retrieval and inference code, F , along with M on DEV.

I2: Authorization. CSM requests permission from PVD to use M on DEV. If approved, PVD issues an authorization token T_{CSM} on M ’s identifier ID_M , encoding CSM’s usage policy on M .

I3: Inference. CSM invokes inference on DEV by presenting T_{CSM} along with the input In_F . Upon successful validation, DEV executes

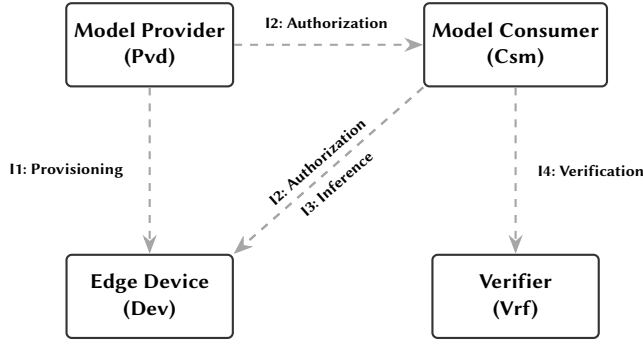


Figure 2: Interactions between PVD, DEV, CSM, and VRF.

$F(In_F)$ with M , producing an inference result Out_F along with a proof T_{proof} .

I4: Proof Verification. CSM sends Out_F and T_{proof} to VRF, which validates the authenticity of Out_F with respect to the expected F and T_{proof} , and accepts Out_F if verification succeeds.

3.2 Threat Model

We consider two adversary cases $\mathcal{A}_M$ and $\mathcal{A}_I$ with different goals.

The goal of $\mathcal{A}_M$ is to extract M . $\mathcal{A}_M$ can compromise DEV (except for its TEE-protected code and data), CSM, and VRF. $\mathcal{A}_M$ cannot compromise PVD who already possesses M . By compromising DEV, $\mathcal{A}_M$ gains full control over its communication channels, enabling message interception, modification, replay, and injection. It also controls all untrusted software on DEV, with the ability to read, write, and execute arbitrary code and data in the Normal World. $\mathcal{A}_M$ also has physical access to DEV, allowing it to extract sensitive information (e.g., a DNN model) that is stored unprotected in DEV's Flash memory (e.g., via Flash dump).

For $\mathcal{A}_I$, the goal is to convince VRF to accept a forged inference result Out_F , that is: one that was not produced by the timely execution of the correct inference software on DEV using appropriate model parameters and external inputs (if any). $\mathcal{A}_I$ can compromise DEV (except for TEE-protected state), and CSM to achieve its goal.

Assumptions. We assume PVD is trusted: as it already possesses M , it has no incentive to act as $\mathcal{A}_M$. Conversely, being the model provider, it can already choose M , gaining no advantage from behaving as $\mathcal{A}_I$.

DEV is assumed to be equipped with a TrustZone-M TEE, whose Secure World code is trusted and free of vulnerabilities. This assumption is corroborated by our design principle of avoiding a potentially large inference function F in the Secure World. Following the typical TrustZone-M trust model [33], we assume an underlying secure boot process that guarantees the integrity of Secure World code at boot time, as well as secure persistent storage restricted to Secure World accesses [14]. To protect against attacks based on Direct Memory Access (DMA) controllers, we assume that DEV is equipped with a security DMA controller capable of specifying DMA access permissions for memory regions, such as the Global TrustZone Controller (GTZC) available on our NUCLEO-L552ZE-Q prototype board.

The trusted PVD is responsible for ensuring F correctness, i.e., it correctly retrieves input data from properly configured peripherals and performs inference without leaking sensitive information (i.e., M). However, given its size, there is still a chance for PVD to make mistakes and introduce vulnerabilities in F . In such a case, while it is not possible to guarantee inference integrity, VECODI ensures that the Secure World remains unaffected (in contrast to the alternative of housing F in the Secure World).

Out-of-Scope Attacks. Following TrustZone-M's standard trust model [33], we do not consider hardware side-channel attacks, invasive hardware modifications or denial-of-service attacks.

3.3 Goals

Under the above system and threat models, VECODI aims to achieve:

G1 (Model Confidentiality): $\mathcal{A}_M$ should not learn any information about M beyond what PVD allows. This goal consists of two sub-goals:

G1-1 (Direct Leakage Protection): Even with compromised DEV, $\mathcal{A}_M$ cannot learn anything about M , except for Out_F and T_{proof} .

G1-2 (Indirect Leakage Protection): Since Out_F may still leak information about M (e.g., via model extraction attacks), $\mathcal{A}_M$ must not be able to invoke F more than what PVD explicitly authorizes.

G2 (Verifiable Inference): An adversary $\mathcal{A}_I$ must not be able to trick VRF into accepting invalid inference results. It consists of two sub-goals:

G2-1 (Authenticity): VRF must be able to detect invalid pairs (Out_F, T_{proof}) submitted by $\mathcal{A}_I$. In particular, this happens when: (1) Out_F is forged without executing F , (2) Out_F is produced by executing F on a different DEV, (3) Out_F is produced by executing F on the right DEV but using a different model than M , or (4) Out_F results from an incomplete execution of F on DEV or from the execution of code different from F .

G2-2 (Privacy): To preserve DEV owner's privacy, VRF must be able to validate (Out_F, T_{proof}) without access to the inference input, which may contain sensitive data.

Remark. The inference input is distinct from In_F . The latter denotes a public input to F , containing only non-sensitive information (e.g., preprocessing parameters or peripheral configuration settings), hence not critical to the DEV owner's privacy. In contrast, the inference input is not a part of In_F ; it is acquired internally during execution through sensor-reading operations and subsequently passed to the inference pipeline within F .

G3 (Low Overhead): To accommodate DEV's limited resources, VECODI must incur minimal overhead on DEV. We consider three types of overhead:

G3-1 (TCB): The increase in Secure World TCB size must be small and independent of F , allowing the same TCB to support arbitrary implementations of input retrieval and inference.

G3-2 (Memory): VECODI should minimize memory usage by avoiding duplication of code or model data across Secure and Normal Worlds whenever possible.

G3-3 (Latency): VECoDI must minimize runtime overhead during inference.

4 Design Overview

4.1 Challenges

Simultaneously achieving **G1** to **G3** is challenging. To illustrate this, we focus on a subset of these: **G1-1** and **G3**. Even under this simplification, we show that commonly used designs in the literature – **D1**, **D2**, and **D3** – fail to satisfy both of them. Since VECoDI also targets more goals, it faces even more challenges.

D1: Full Normal World Execution. We start with a strawman approach that executes F entirely in *Normal World*. While this approach naturally satisfies **G3** and is easy to deploy, it provides no security guarantees: $\mathcal{A}_M$ can trivially violate **G1-1**, e.g., by extracting M from DEV’s flash/RAM while $\mathcal{A}_I$ can steal the attestation key in Normal World, resulting in a successful forgery of T_{proof} .

D2: Fully Secure World Execution. At the other end of the spectrum, F can be placed and executed entirely in the Secure World. To protect M at rest, PVD can provision it in encrypted form (during **I1: Provisioning**), which is then decrypted and loaded into secure RAM for execution by F post-boot. This satisfies **G1-1**, not **G3**. In particular, **G3-1 (TCB)** is violated, as the Secure World must include F , which is application-specific (e.g., dependent on input sensors and inference implementations). Additionally, **G3-2 (Memory)** is strained, since DEV must store both an encrypted version of M in flash and a plaintext version in secure RAM. Finally, this design limits the use of emerging ML hardware accelerators (e.g., microNPUs [24], Edge TPUs [46]), which are typically exposed to and managed by the Normal World [40], thereby hindering potential performance gains.

D3: Split Inference Across Worlds. A middle-ground partitions M into private (M_{pri}) and public components (M_{pub}). Here, M_{pri} is provisioned in encrypted form and executed in Secure World, similar to **D2**; M_{pub} is managed and executed in the Normal World, as done in **D1**. This design is commonly used in prior TEE-based work [5, 19, 25] to achieve **G1-1** on potentially untrusted devices. Since M_{pri} is typically much smaller than M_{pub} , this design reduces secure memory requirements compared to **D2**, partially addressing **G3-2 (Memory)**. However, it does not resolve **G3-1 (TCB)**, as Secure World must still host F to ensure correct usage of M_{pri} . Moreover, **G3-3 (Latency)** can be negatively impacted due to frequent world switches between Secure and Normal Worlds, with overhead depending on the partitioning strategy, e.g., approaches like TEESlice [19], which distribute private components across multiple layers, can incur substantial switching costs. Finally, this design still limits the use of emerging low-cost hardware accelerators for executing M_{pri} .

4.2 High-Level Intuition

From **D1–D3**, we observe that **G3** is achieved only by **D1**. This suggests that extending **D1** with stronger security mechanisms could enable us to achieve both **G1-1** and **G3**.

Guided by this insight, we propose a new abstraction, SHANGRI-LA, which executes F in Normal World, inheriting **D1**’s benefits for achieving **G3**. To enforce **G1-1**, we introduce a small, F-agnostic component in the Secure World that elevates SHANGRI-LA privilege

Table 1: Memory access control of TrustZone-M domains with our introduction of SHANGRI-LA; C and D indicate code and data, respectively, while PrivD and PubD refer to private and public data. RO indicates read-only access.

	Normal World		SHANGRI-LA			Secure World	
	C	D	C	PrivD	PubD	C	D
Normal World	✓	✓	RO	×	RO	×	×
SHANGRI-LA	✓	✓	✓	✓	✓	×	×
Secure World	✓	✓	✓	✓	✓	✓	✓

during its execution, placing its privilege between Normal and Secure Worlds.

With this new privilege, Normal World cannot access or tamper with SHANGRI-LA’s code and data, ensuring **G1-1**. At the same time, SHANGRI-LA is prevented from interfering with Secure World. As a result, this design enables third-party developers to deploy isolated code without modifying highly trusted Secure World components. Overall, SHANGRI-LA establishes a new, third, runtime environment on TrustZone-M where the privilege relationships among the three environments are summarized in Table 1.

Composition with Cryptographic Authorization and Proofs of Execution. To achieve complete protection, we compose SHANGRI-LA to support **G1-2** and **G2** without sacrificing **G3**. Specifically, SHANGRI-LA is extended with two features: (1) an authorization mechanism that allows a trusted party to control who can invoke SHANGRI-LA and how often, thereby enforcing **G1-2** (assuming F correctly performs input retrieval and inference over M), and (2) a proof-of-execution mechanism that enables Secure World to produce verifiable evidence that isolated code within SHANGRI-LA executed correctly and that a given output corresponds to that execution.

5 SHANGRI-LA Isolation Abstraction: A Third Runtime Environment on TrustZone-M

Here, we describe SHANGRI-LA, starting with its high-level lifecycle and then detailing the Secure-World APIs used to orchestrate its lifecycle state.

5.1 Lifecycle of SHANGRI-LA

The lifecycle of SHANGRI-LA (Figure 3a) consists of three states: Non-Exist, Inactive, and Active, with transitions controlled by four Secure World APIs.

On a new DEV, SHANGRI-LA is in the Non-Exist state. During provisioning, Provision API is called to install the necessary SHANGRI-LA metadata (e.g., code/data hash and keys), while Authorize configures access control policies such as permitted users and usage limits. These steps prepare SHANGRI-LA for deployment but do not instantiate it, so it remains in the Non-Exist state.

To create an executable instance, the Normal World invokes Create. This validates SHANGRI-LA’s code and data with the stored metadata, initializes and assigns it the access privileges defined in Table 1, and transitions it to the Inactive state, where it is ready to run. Invoking Execute transitions SHANGRI-LA to the Active

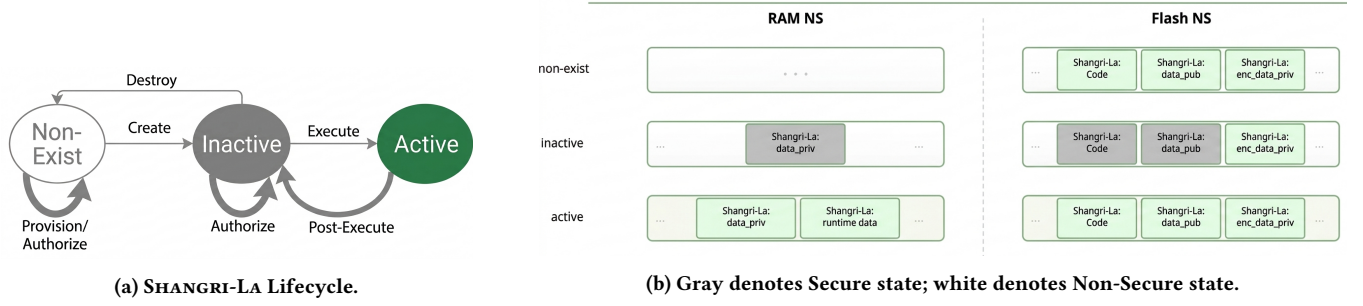


Figure 3: Overview of SHANGRI-LA lifecycle (a) and its corresponding TrustZone-M memory state (b).

state, during which it executes F . Upon completion, it returns to the Inactive state. This enables repeated executions by alternating between Inactive and Active via Execute. While in the Inactive state, Authorize can be invoked to update SHANGRI-LA authorization policy without requiring re-creation.

When the SHANGRI-LA instance is no longer needed, Destroy can be invoked to erase its sensitive state and release its memory back to the Normal World, returning to the Non-Exist state.

Next, we describe how aforementioned Secure World APIs are designed/implemented using TrustZone-M.

5.2 Secure-World APIs

To orchestrate the SHANGRI-LA lifecycle, we maintain its context in secure persistent storage and expose a set of Secure-World APIs that allow the Normal World to trigger controlled state transitions without directly accessing or manipulating SHANGRI-LA code and data.

As a basic form, we provide Provision, Create, Execute, and Destroy, which together ensure that SHANGRI-LA adheres to the privilege model in Table 1. We further introduce two optional extensions: (1) an Authorize API that allows DEV owner to specify which users may access SHANGRI-LA and how many times it can be invoked, and (2) an input flag to Execute that enables generation of a proof of execution, allowing an external verifier to validate the authenticity of the output.

All APIs are summarized in Algorithm 1, while the corresponding TrustZone-M security states of SHANGRI-LA memory across its lifecycle are illustrated in Figure 3b.

5.2.1 Provision (Line 2-5). Unlike the other APIs, Provision is only accessible to a trusted entity, e.g., during device manufacturing or via a Secure-World update mechanism that verifies the integrity of the SHANGRI-LA configuration prior to its installation.

It takes as input the SHANGRI-LA owner's public key pk_o and a public identifier s_id describing the SHANGRI-LA instance, containing its code F , entry point $entry$, and a data identifier d_id that captures both public data $data_{pub}$ and encrypted private data enc_data_{priv} (if any). If enc_data_{priv} is present, a decryption key k_{dec} is also provided. Optionally, the API accepts a user public key pk_u and an access limit $limit$, specifying which user is authorized to invoke F (which executes within SHANGRI-LA) and how many times it may be executed.

Upon invocation, it computes a hash of s_id to serve as a unique identifier for the SHANGRI-LA instance, initializes its Secure-World context (CTX) with the provided input, sets the instance's usage counter to 0, and places the instance in the Non-Exist state.

5.2.2 Authorize (Line 6-12). This API updates the access policy of a provisioned SHANGRI-LA. It takes as input a user public key pk_u , a new limit $limit$, a unique SHANGRI-LA identifier H_{s_id} , and an authorization token T_o over $(pk_u, limit, H_{s_id})$ produced by the SHANGRI-LA owner. Upon invocation, it verifies that a SHANGRI-LA instance identified by H_{s_id} has been provisioned and validates T_o using pk_o . If both checks succeed, it updates the instance's context with pk_u and $limit$ accordingly. This API enables the SHANGRI-LA owner to dynamically change the authorized user and adjust the number of allowed invocations for that user.

5.2.3 Create (Line 13-20). This API initializes a SHANGRI-LA instance from its provisioned configuration, taking s_id as input. Then, it retrieves the memory regions corresponding to the code F , public data $data_{pub}$, and encrypted private data enc_data_{priv} from the instance's context. It then uses the TrustZone-M's SAU and security DMA controller (recall Section 2 and 3.2) to ephemerally mark the code and $data_{pub}$ regions as Secure, protecting them from both Normal World access and Non-Secure DMA. If encrypted private data exists, the API allocates a region $data_{priv}$, ephemerally marks it as Secure, and decrypts enc_data_{priv} using k_{dec} into this region. To prevent unauthorized reads by external peripherals, the API places $data_{priv}$ in a Non-Secure RAM region that does not overlap with any memory-mapped peripheral regions. Finally, it sets the SHANGRI-LA lifecycle state to Inactive, indicating that the instance has been populated with code and data and is ready for execution. As illustrated in Figure 3b, after this call, all SHANGRI-LA code and data are in the Secure state, preventing access or modification by the Normal World.

Remark. Although F and its data are temporarily marked as Secure, F is never executed in this state. Instead, it is atomically restored to Non-Secure immediately before execution (see Execute below), ensuring that the SHANGRI-LA state cannot be altered between its definition and execution and at the same time maintaining isolation between F and the Secure World TCB.

5.2.4 Execute (Line 21-40). This API performs a single F invocation in SHANGRI-LA. It takes as input a user message containing

Algorithm 1: SHANGRI-LA Secure World APIs

```

1 Context:  $CTX : H_{SHANGRI-LA} \rightarrow$   

    $(k_{dec}, pk_o, pk_u, data\_id, entry, usage, limit, state)$ 

2 Function Provision( $k_{dec}, pk_o, pk_u, s\_id, limit$ )
3    $H_{s\_id} \leftarrow \text{Hash}(s\_id)$ 
4   parse ( $data\_id, entry$ ) from  $s\_id$ 
5    $CTX[H_{s\_id}] \leftarrow$   

    $(k_{dec}, pk_o, pk_u, data\_id, entry, 0, limit, \text{Non-Exist})$ 

6 Function Authorize( $pk_u, limit, H_{s\_id}, T_o$ )
7   if  $H_{s\_id} \notin CTX$  then
8     abort
9   retrieve  $pk_o$  from  $CTX[H_{s\_id}]$ 
10  if  $limit \leq CTX[H_{s\_id}].limit$  or not Verify( $pk_o, T_o,$   

    $pk_u || limit || H_{s\_id}$ ) then
11    abort
12   $CTX[H_{s\_id}].\{pk_u, limit\} \leftarrow \{pk_u, limit\}$ 

13 Function Create( $s\_id$ )
14   $H_{s\_id} \leftarrow \text{Hash}(s\_id)$ 
15  if  $H_{s\_id} \notin CTX$  then
16    return failure
17  parse ( $F, data_{pub}, enc\_data_{priv}$ ) from  $s\_id$ 
18  mark ( $F, data_{pub}, data_{priv}$ ) as Secure
19   $data_{priv} \leftarrow \text{AuthDec}(CTX[H_{s\_id}].k_{dec}, enc\_data_{priv})$ 
20   $CTX[H_{s\_id}].state \leftarrow \text{Init}$ 

21 Function Execute( $u, In, H_{s\_id}, proof, T_u$ )
22  if  $H_{s\_id} \notin CTX$  then
23    abort
24   $(pk_o, pk_u, data\_id, entry, usage, limit, state) \leftarrow$   

    $CTX[H_{s\_id}]$ 
25  if  $state \neq \text{Init}$  or  $usage \geq limit$  or  $u \neq usage + 1$  then
26    abort
27  if not Verify( $pk_u, T_u, u || In || H_{s\_id} || proof$ ) then
28    abort
29  disable interrupts
30   $CTX[H_{s\_id}].state \leftarrow \text{Active}$ 
31  allocate SHANGRI-LA stack
32  mark ( $F, data_{pub}, data_{priv}$ ) as Non-secure
33   $Out_F \leftarrow \text{execute entry}(In)$ 
34  erase stack
35  mark ( $F, data_{pub}, data_{priv}$ ) as Secure
36   $CTX[H_{s\_id}].\{usage, state\} \leftarrow \{u, \text{Inactive}\}$ 
37  enable interrupts
38  if  $proof$  then
39     $T_{proof} \leftarrow \text{Sign}(sk_{DEV}, F || u || data\_id || In_F || Out_F ||$   

    $pk_o)$ 
40  return ( $Out_F, T_{proof}$ )

41 Function Destroy( $H_{s\_id}$ )
42  if  $H_{s\_id} \notin CTX$  then
43    abort
44  erase  $data_{priv}$ 
45  mark ( $F, data_{pub}, data_{priv}$ ) as Non-secure
46   $CTX[H_{s\_id}].state \leftarrow \text{Non-Exist}$ 

```

the post-execution usage counter u , F input In_F , $H(s_id)$, and a signature T_u of this message produced by the user.

Upon invocation, it looks up a SHANGRI-LA instance from H_{s_id} and retrieves its context. It further checks that: (1) the instance is in the Inactive state, (2) the provided counter u is consistent with the current usage (i.e., $u = usage + 1$), (3) the invocation does not exceed $limit$ and (4) authenticity of T_u using pk_u .

If all checks succeed, Secure World prepares execution of F in SHANGRI-LA as follows: it disables interrupts, sets the SHANGRI-LA state to Active, and allocates a dedicated stack at a fixed location (that does not overlap with memory-mapped peripheral regions) in Normal World. Next, Secure World marks F, $data_{pub}$ and $data_{priv}$ as Non-Secure (see Figure 3b) and calls F entry point ($entry$) to execute F with input In_F in the Normal World. Once F execution completes, control returns to Secure World (this is achieved by requiring an NSC at the end of F). The Secure World then erases the SHANGRI-LA stack, restores SHANGRI-LA code and data to Secure state, increments the usage counter, and sets the lifecycle state back to Inactive before re-enabling interrupts.

If a proof of execution is required, Secure World generates T_{proof} by signing ($F, u, data_id, In_F, Out_F$) using the DEV-bound attestation key sk_{DEV} . Finally, it returns (T_{proof}, Out_F) to the caller.

5.2.5 Destroy (Line 41-46). This API tears down a SHANGRI-LA instance H_{s_id} . Upon invocation, Secure World erases all sensitive data in $data_{priv}$. It then marks F, $data_{pub}$ and $data_{priv}$ regions as Non-Secure, releasing the memory back to the Normal World. Finally, it sets the SHANGRI-LA lifecycle state to Non-Exist.

5.3 SHANGRI-LA Security Analysis

We argue that SHANGRI-LA isolation abstraction achieves the following security properties:

P1: Privilege Separation. We show that the four core APIs – Provision, Create, Execute (without proof-of-execution), and Destroy – are sufficient to enforce the privilege model in Table 1.

During Execute, F runs atomically with interrupts disabled, and control is not returned to Normal World until both F and Execute complete. This prevents Normal World from interrupting execution to access $data_{pub}$ or $data_{priv}$ (which are in Non-Secure state) while SHANGRI-LA is in the Active state.

A compromised Normal World may attempt to access SHANGRI-LA code and data outside the Active state. When SHANGRI-LA is in the Inactive state, such access is prevented by the SAU, as all SHANGRI-LA memory regions are marked Secure (Figure 3b). DMA-based attacks are mitigated in the same way, since the Secure/Non-Secure memory attributes also apply to DMA accesses. The remaining opportunity is when SHANGRI-LA is in the Non-Exist state. In this state, F and $data_{pub}$ reside in Non-Secure flash and may be read or modified. However, any tampering is detected during Create, as the recomputed H_{s_id} (over F and $data_{pub}$) will not match the provisioned value, preventing SHANGRI-LA from being created. For $data_{priv}$, confidentiality is preserved since it exists only in encrypted form in the Non-Exist state.

A subtle case arises if the device resets while SHANGRI-LA is in the Active or Inactive state, where $data_{priv}$ or its derived runtime state may persist in Non-Secure RAM. To address this, we

propose either: (i) extending Secure-World initialization to clear all Non-Secure RAM at boot, or (ii) having Execute record the stack region in its context and extending Secure-World initialization to detect incomplete executions and securely erase the corresponding memory.

We note that SHANGRI-LA always lives/executes in the Non-Secure state and therefore cannot access or interfere with higher-assurance Secure-World components. F code and data are only marked as Secure in between Create and Execute, but never executed in this state, being brought back to Non-Secure upon Execute invocation. This dynamic remarking indeed is what enables SHANGRI-LA existence and an intermediate security state between Normal and Secure Worlds. At the same time, executing in the Non-Secure state allows SHANGRI-LA to access and reuse existing Non-Secure code when needed. As a result, these satisfy the privilege model in Table 1.

P2: Authorization of SHANGRI-LA Invocation. We show that, with the addition of the Authorize API, only authorized users can invoke F in SHANGRI-LA, and they cannot exceed the usage limit specified by the owner.

An unauthorized user cannot execute F because their public key is not registered in the SHANGRI-LA (Secure World-protected) context. The Execute API requires a user message signed by the authorized user and verifies it against the authorized user's public key pk_u . Without access to the corresponding private key, an adversary cannot produce a valid signature and thus cannot run a SHANGRI-LA instance.

Furthermore, Authorize updates the authorized user and usage limit only upon receiving a valid owner-signed message (verified using pk_o). The usage counter is restricted to Secure World and checked during Execute to ensure that each invocation is fresh and does not exceed the specified limit. This prevents both overuse and replay attacks.

P3: PoX Unforgeability. Finally, when the proof-of-execution feature is enabled ($proof = 1$ in Execute), we show that a valid T_{proof} can only be produced if F is executed correctly within SHANGRI-LA. We adopt the notion of correct execution from prior work [30], where F must execute completely from its entry point to its exit point without interruption and the value of Out_F must be determined by this execution.

T_{proof} is generated inside Secure World via the Execute API using sk_{DEV} and covers $(F, u, data_id, In_F, Out_F)$. Since sk_{DEV} is not accessible to the Normal World, an adversary cannot forge a valid T_{proof} . Consequently, a valid T_{proof} can only be obtained by successfully invoking Execute, which runs F with input In_F inside SHANGRI-LA to produce the corresponding output Out_F . Any attempt to modify In_F or Out_F results in an invalid proof, as both are cryptographically bound in T_{proof} . Also, interrupting F execution to induce an incomplete execution is not possible due to interrupts being disabled during F execution. A reset during Execute may cause F to terminate prematurely; however, in this case, no T_{proof} is produced, since it is generated only upon successful completion of Execute. In all cases, an adversary cannot forge T_{proof} without having to execute F inside SHANGRI-LA with the correct In_F and produced Out_F .

6 VECODI: Edge Inference Stakeholder Interactions

Building on the SHANGRI-LA abstraction and its Secure-World APIs (Section 5), we present VECODI, a protocol that realizes secure edge inference interactions **I1–I4** defined in Section 3. The protocol is summarized in Algorithm 2.

Algorithm 2: VECODI Protocol

```

1 I1: Provisioning (PVD → DEV)
   // Executed by Pvd
2    $(M_{pub}, M_{pri}) \leftarrow \text{TDSP-Partition}(M)$ 
3    $s\_id \leftarrow (F, entry, M_{ID}, M_{pub}, \text{AuthEnc}(k_{dec}, M_{pri}))$ 
   // Executed on Dev by a trusted entity
4   Install F,  $data_{pub}$ ,  $enc\_data_{priv}$  in Normal World
5   Provision( $k_{dec}, pk_{PVD}, pk_{CSM}, s\_id, limit$ )

6 I2: Authorization (PVD → CSM → DEV)
   // Executed by Pvd, after verifying Csm's request
7    $T_{PVD} \leftarrow \text{Sign}(sk_{PVD}, pk_{CSM} \parallel limit \parallel H_{s\_id})$ 
   // Executed on Dev
8   Authorize( $pk_{CSM}, limit, H_{s\_id}, T_{PVD}$ )

9 I3: Inference (CSM ↔ DEV)
   // Executed by Csm
10   $T_{CSM} \leftarrow \text{Sign}(sk_{CSM}, u \parallel In_F \parallel H_{s\_id} \parallel proof)$ 
   // Executed on Dev
11   $(Out_F, T_{proof}) \leftarrow \text{Execute}(u, In_F, H_{s\_id}, proof, T_u)$ 

12 I4: Proof Verification (CSM → VRF)
13  if not Verify( $pk_{DEV}, T_{proof}, F \parallel u \parallel M_{ID} \parallel In_F \parallel Out_F \parallel pk_{PVD}$ ) or not
    IsValid( $F, M_{ID}, In_F, pk_{PVD}$ ) then
14    reject  $Out_F$ 
15  else
16    accept  $Out_F$ 

```

6.1 I1–I4 in VECODI

I1: Provisioning. PVD prepares the software F with entry point *entry*. F is a self-contained binary that bundles the input-retrieval code (e.g., a sensor driver), the inference library, and the glue code connecting them, executing both back-to-back, i.e., it first retrieves the inference input directly from DEV's hardware peripherals, then immediately runs DNN inference on it. Given a DNN model M , PVD assigns its ID as M_{ID} and then applies a TDSP partitioning strategy to split it into private layers M_{pri} and public layers M_{pub} . It then generates a symmetric key k_{dec} and uses it to encrypt M_{pri} .

Using s_id , PVD installs F, M_{pub} , and encrypted M_{pri} in the Normal World of DEV, while provisioning the corresponding SHANGRI-LA metadata to the Secure World via the Provision API. Besides s_id and k_{dec} , this metadata also includes PVD's public key pk_{PVD} , and optionally the authorized user's public key pk_{CSM} together with the usage limit *limit*.

Note that VECODI is generic and supports both using or not using TDSP. In the latter case M_{pri} is the entire model and M_{pub} is empty.

I2: Authorization. CSM requests permission from PVD to invoke inference on DEV. If approved, PVD issues a signed authorization token T_{CSM} that grants the user with public key pk_{CSM} access to the SHANGRI-LA instance identified by H_{s_id} with a usage limit $limit$.

CSM forwards T_{CSM} to DEV, which invokes the Authorize API to update the access policy of the corresponding SHANGRI-LA instance. Upon successful verification of T_{CSM} , the Secure World registers pk_{CSM} and $limit$ for H_{s_id} .

I3: Inference. To perform inference, CSM first invokes the Create API to instantiate the SHANGRI-LA instance identified by H_{s_id} . It then constructs a request containing the next usage counter u , the input In_F , the identifier H_{s_id} , and a flag $proof = 1$ indicating that a proof of execution is required. This request is signed using sk_{CSM} to produce an inference token T_{proof} .

CSM submits the request and T_{proof} to the Execute API. Upon successful verification, Execute runs the corresponding function F within SHANGRI-LA, which retrieves input data (e.g., from a sensor) and performs inference using M , producing an output Out_F . Since $proof = 1$, this invocation returns a proof of execution T_{proof} .

I4: Proof Verification. CSM forwards (Out_F, T_{proof}) to VRF. VRF checks that T_{proof} is a valid token under pk_{DEV} over the expected pk_{PVD} , u , F , M_{ID} , In_F , and Out_F .

6.2 VECODI Security Analysis

We show that VECODI satisfies **G1** and **G2**. Our arguments build on the security properties of SHANGRI-LA established in Section 5.3.

G1-1 (Direct Leakage Protection). The DNN model M is partitioned into M_{pub} and M_{pri} , where M_{pub} is deliberately set as public by PVD. Thus, **G1-1** concerns only the protection of M_{pri} .

In VECODI, M_{pri} is treated as private data within SHANGRI-LA and therefore inherits **P1: Privilege Separation** that guarantees the access control guarantees in Table 1. In particular, untrusted Normal World on DEV cannot access M_{pri} in plaintext. Outside DEV, the only point at which M_{pri} is handled in plaintext is during **I1: Provisioning** by PVD. As $\mathcal{A}_M$ excludes PVD, $\mathcal{A}_M$ cannot directly observe M_{pri} , satisfying **G1-1**.

G1-2 (Indirect Leakage Protection). Indirect leakage through overused or unauthorized inference queries is mitigated through **I2: Authorization** in VECODI. Therein, CSM must obtain a valid T_{CSM} from PVD, which will be enforced on DEV by the Authorize API before any inference can be executed through Execute. As a result, VECODI inherits **P2: Authorization of SHANGRI-LA Invocation**, preventing unauthorized $\mathcal{A}_M$ from performing inference on M more than PVD's specified usage limit.

G2-1 (Authenticity). In VECODI, **I3: Inference** requires invoking the Execute API, thereby inheriting **P3: PoX Unforgeability** of SHANGRI-LA. In particular, any attempt by $\mathcal{A}_I$ to forge an inference result Out_F or its corresponding proof T_{proof} will be detected by VRF. Freshness is ensured through the inclusion of u in T_{proof} , where

u serves as a nonce to prevent replay attacks. VRF can obtain an up-to-date value of u from a trusted source, such as PVD.

Moreover, since F encapsulates both input retrieval and inference, a valid T_{proof} guarantees that the inference input is obtained without being tampered with and that F executes completely without interruption. The acquisition of local DEV sensor inputs is implemented in F code and thus implicitly checked by checking F proper execution. As T_{proof} also covers authentic M_{ID} and pk_{PVD} (protected on DEV through Secure World's secure storage), VRF can determine whether the inference is performed using the right DNN model M . In sum, VECODI assures that Out_F is the authentic result of executing F on input In_F using the model identified by M_{ID} , as provisioned by PVD. Thus, **G2-1** is fulfilled.

G2-2 (Privacy). In **I4: Proof Verification**, verification relies on public information, including F , In_F , M_{ID} , pk_{PVD} , Out_F and T_{proof} . Importantly, the raw inference input is neither revealed nor required during verification, as it is acquired and processed internally within SHANGRI-LA, which is also not exposed to DEV's Normal World. Hence, VECODI preserves the privacy of DEV inputs used for inference, satisfying **G2-2**.

7 Evaluation

We implement VECODI on a NUCLEO-L552ZE-Q development board [35], which integrates an ARM Cortex-M33 MCU running at 110MHz with 512KB of Flash memory and 256KB of SRAM, where 192KB is used for the Normal World (hereafter referred to as Non-Secure or NS RAM) and the remaining 64KB is reserved for the Secure World (S RAM).

Secure World Setup. The Secure World runs Trusted Firmware-M (TF-M) [41], a reference Secure Processing Environment for Armv8-M systems. We extend TF-M to implement VECODI's Secure-World APIs on DEV. Specifically, we use the TF-M Crypto Partition to provide cryptographic services and expose them to the Normal World via NSC veneers. For digital signatures on DEV, we rely on TF-M's ECDSA APIs (PSA_ALG_ECDSA) for signing and verification. At boot, TF-M configures five of the eight SAU regions. Regions 0 and 1 map the Flash and SRAM assigned to the Normal World. Regions 2-4 are reserved for NSC veneers, peripherals, and platform-specific registers, and are left unchanged by our implementation.

When an SHANGRI-LA instance enters the Inactive state via the Create API, it dynamically reconfigures additional SAU regions to isolate its code and data. For SRAM, Region 1 is disabled, while Regions 6 and 7 are configured to cover all non-secure RAM except the private-data region, which thus becomes part of Secure state by default. For Flash, Region 0 covers the area preceding the SHANGRI-LA code and public data, and Region 5 covers the following region up to Secure Flash. Upon transitions to other lifecycle states, the Secure-World APIs restore the SAU configuration to its boot-time defaults.

SHANGRI-LA Setup. In our experiments, F running inside SHANGRI-LA performs two tasks: (1) acquiring an inference input and (2) executing inference. For (1), we use a simple routine that reads an image from Flash memory to establish a lower-bound timing baseline. A real-world sensor pipeline using an external

Table 2: TCB comparison between VECODI and the baseline.

VECODI API	LOC	% of Baseline (34,865 LOC)
Authorize	355	1.02%
Create	153	0.44%
Execute	481	1.38%
Destroy	23	0.07%
Total TCB	1012	2.91%

camera is presented later in Section 7.4. For (2), we use the CMSIS-NN library [18], which provides optimized DNN kernels with low memory overhead for Arm Cortex-M processors. As our target model, we adopt ResNet [37], a lightweight architecture widely used in prior edge-inference work [5, 13, 21]. Following CMSIS-NN guidelines, we apply fixed-point quantization, resulting in a model size of 76.375KB. With ResNet, VECODI supports arbitrary TSDP partitioning strategies that split the model into M_{pub} and M_{priv} . In our evaluation, we follow [5], designating the final three layers of ResNet as private to reduce the risk of model-stealing attacks. These layers form M_{priv} and are provisioned in encrypted form in Flash during **I1: Provisioning**, while the remaining layers M_{pub} remain unprotected. As a result, the full model size is $|M| = 76.375\text{KB}$, with $|M_{priv}| = 38.625\text{KB}$ and $|M_{pub}| = 37.75\text{KB}$. In our setup, Provision is performed offline by manually installing F, emulating secure factory provisioning; thus, there are no runtime results to report for provisioning.

Baseline. To evaluate VECODI's overhead, we compare against a baseline where F runs entirely in the Secure World and produces inference results within the trusted environment. To ensure authenticity, outputs are signed with a DEV-bound key, providing security guarantees comparable to VECODI. This setup corresponds to **D2** in Section 4.1.

7.1 Results for G3-1 (TCB)

The baseline runs F in the Secure World, so the TCB includes F, which is dominated by the CMSIS-NN library (34,865 LOC). The remaining code handles request validation and output signing, resulting in a total TCB of 34,865 LOC.

Table 2 reports the Secure-World TCB of DEV, broken down by component and compared to the baseline. As expected, Execute has the largest TCB contribution (481 LOC) due to validation checks, memory management, and PoX. In contrast, Destroy has the smallest TCB (23 LOC), as it only handles memory erasure.

Compared to the baseline, VECODI reduces the TCB from 34,865 LOC to 1012 LOC (2.91% of the baseline). Further, the TCB on DEV is agnostic to F, and hence to sensor reading driver and inference library, making it unchanged across different combinations of sensor-input reading implementation and model architectures/DNN library (e.g., TensorFlow-Lite [6]). This contrasts with the baseline approach, where the TCB grows with a new F function, making it hard to realize for multiple F variants. This satisfies **G3-1 (TCB)**.

7.2 Results for G3-2 (Memory)

In the baseline, the full model M executes in the Secure World and is treated as confidential. To prevent extraction from Flash (Flash

Table 3: Inference latency of VECODI vs. baseline using the ResNet model, averaged over 100 runs.

NS baseline	VECODI	PoX/Signing	Overhead
32.083 ms	32.161 ms	0.16 s	0.078 ms (0.24%)

dump), M must be stored encrypted and decrypted into Secure RAM at runtime, creating two copies: ciphertext in Flash and plaintext in Secure RAM. In the test case, this requires 76.375 KB of extra Secure RAM.

In contrast, VECODI partitions M into M_{pub} and M_{priv} . Only M_{priv} is stored encrypted in Flash, while M_{pub} remains in plaintext and can be accessed directly from Flash at runtime. During execution, VECODI instantiates an SHANGRI-LA that decrypts M_{priv} into SHANGRI-LA-protected Non-Secure RAM. As a result, only M_{priv} is duplicated, whereas M_{pub} exists as a single Flash-resident copy.

As a result, VECODI substantially reduces runtime memory duplication and moves the duplicated memory footprint away from Secure RAM. Unlike the baseline, the duplicated model portion M_{priv} resides in NS RAM (securely protected by SHANGRI-LA) rather than in Secure RAM, making more Secure-World memory available to other security-critical services. Also, since VECODI is agnostic to the partitioning strategy, deployments can further reduce memory overhead by selecting strategies that minimize the size of M_{priv} . This satisfies **G3-2 (Memory)**.

7.3 Results for G3-3 (Latency)

For **G3-3 (Latency)**, we measure the end-to-end inference latency of VECODI's Execute. All results reported are obtained as the average over 100 executions. As discussed earlier, in the baseline case, the full decrypted model M requires about 76.375 KB. However, Secure RAM is limited to 64 KB, making the baseline infeasible for direct comparison. Instead, we use an *NS baseline*, which mimics the original baseline design but loaded in the Normal World, where a larger RAM size is available (192 KB).

Table 3 reports the inference latency of VECODI (via the Execute API) and the NS baseline. Compared to the baseline, VECODI adds only 0.078 ms of latency, for a total runtime of 32.161 ms. This runtime includes SAU/DMA protection region reconfiguration, SHANGRI-LA stack setup and erasure, and state management. The PoX computation adds an additional 0.16 s to generate the cryptographic signature/proof sent to VRF. However, in the baseline, the inference output must also be signed by the Secure World (to at least authenticate DEV), resulting in the same cost also applying to the baseline. Inference is executed more frequently than other operations, making it the most critical. The runtimes of the remaining operations are shown in Table 4, for a fixed M_{priv} size of 39 KB.

As the runtime of Create and Destroy depends on the size of M_{priv} , we further evaluate their runtime across varying M_{priv} sizes. Figure 4 shows that Create latency scales linearly with M_{priv} , increasing from 12.26 ms at 1 KB to 478 ms at 39.5 KB. By comparison, Destroy remains substantially faster (Figure 5), ranging from 0.223 ms to 2.707 ms across the same configurations. In practical deployments, SHANGRI-LA construction and destruction would likely

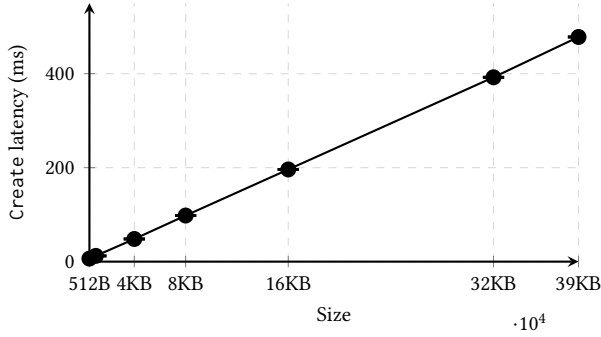


Figure 4: Latency of Create vs. M_{priv} size.

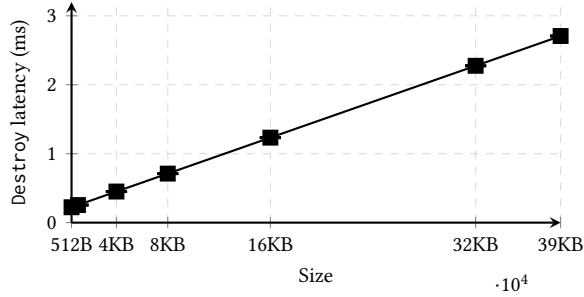


Figure 5: Latency of Destroy vs. M_{priv} size.

Table 4: VeCoDI latency breakdown.

Metric	Authorize	Create	Execute	Destroy
Cycles	262,000	52,611,507	3,537,710	298,277
Latency (ms)	2.382	478.140	32.161	2.712

occur once per session rather than once per inference, making their amortized overhead small.

7.4 Case Study

To validate VeCoDI in a realistic deployment setting, we implement a complete image classification pipeline on the NUCLEO-L552ZE-Q board (including image acquisition via UART from a hardware camera interface). The device receives images over an UART-based camera stream, runs inference using the encrypted ResNet model, and returns classification results. This pipeline is included in our open-source repository.

We follow a role-separated workflow. PVD establishes the session, provides the encrypted model, and authorizes a bounded inference budget. CSM creates the SHANGRI-LA, issues inference requests within the authorized budget, and destroys the SHANGRI-LA once the quota is exhausted. Over 100 sequential inference runs, VeCoDI achieves 48.359 ms latency per inference, compared to 48.281 ms for the NS baseline. At the same time, model parameters and authorization state remain isolated from the Normal World.

Security tests. We further validate VeCoDI against representative attacker actions, consistent with our threat model, by executing one directed negative test per attack type:

Table 5: Comparison with prior TrustZone (TZ)-based work on edge inference (~ means partially-fulfilled).

	TEE	G1-1	G1-2	G2-1	G2-2	G3-1	G3-2	G3-3
Darknetz [25]	TZ-A	✓	~	×	×	×	✓	✓
ShadowNet [36]	TZ-A	✓	×	×	×	×	✓	×
ASGARD [26]	TZ-A	✓	×	×	×	×	×	×
SecureQNN [5]	TZ-M	✓	~	×	×	×	✓	✓
VeCoDI	TZ-M	✓	✓	✓	✓	✓	✓	✓

Memory Access Attacks. We attempt to read Secure SRAM regions holding authorization state (201 bytes) from Non-Secure code, and similarly attempt to read protected Flash regions mapped to SHANGRI-LA code. All cases trigger hardware faults and are blocked by SAU, confirming correct memory isolation.

Protocol Attacks. We test three protocol violations: (i) replaying a previously valid inference request, (ii) issuing an inference request after budget exhaustion, and (iii) forging the session-model binding $H_{s.id}$. All are rejected without modifying Secure state.

These tests confirm that VeCoDI enforces authorization, freshness, and model-identity binding per goals introduced in Section 3.3. Combined with the low latency overhead (Section 7.3), the reduced Secure memory footprint, and the model-independent Secure TCB (Table 2), these results show that VeCoDI is a viable and secure approach for resource-constrained Cortex-M devices.

8 Discussion, Limitations, and Extensions

Multiple Inferences per Execution. Our case study focuses on producing a proof for a single inference while preserving model confidentiality. However, the design can be extended to support verifiable execution over multiple sensor inputs within a single invocation. Rather than invoking Execute repeatedly, F can be written in a way to accept an additional parameter specifying the number of sensor readings to acquire. During execution, F records multiple inputs within the SHANGRI-LA stack. After collecting the required number of inputs, F performs batched inference over the recorded data, producing a vector of outputs. Since SHANGRI-LA generates a proof that binds execution to the correct invocation of F, the resulting proof can also attest that the output vector was produced from the specified batch of inputs under a single execution context. Supporting this extension would additionally require Execute to account for the total number of inferences performed within one invocation when enforcing the authorized usage limit.

Interrupt-driven F. To ensure verifiable execution, VeCoDI currently relies on SHANGRI-LA to disable interrupts during execution of F. As a result, F must not depend on interrupt-driven behavior. While this assumption holds in our case study, it may not apply to applications where F relies on interrupt-triggered sensor inputs. One possible extension is to incorporate an interrupt-compatible proof-of-execution mechanism such as [3]. Doing so permits interrupts during execution by enforcing read-only protection over program memory and the interrupt vector table (IVT), while computing a hash of the IVT snapshot before execution begins. The resulting hash can then be incorporated into T_{proof} or the generated proof to preserve execution integrity despite interrupt handling.

Cryptographic Choice. VECODI protocol relies on public-key cryptography for authentication and proof generation. While this overhead is generally acceptable for PVD, CSM, and VRF, it may be expensive for resource-constrained DEV-s. In VECODI, DEV maintains three keys: pk_{PVD} , pk_{CSM} , and sk_{DEV} . The first two are used to authenticate requests in `Authorize` and `Execute`, respectively, while sk_{DEV} is used to generate T_{proof} . To reduce computational overhead on DEV, VECODI can replace public-key operations with symmetric-key ones. Since PVD controls provisioning, it may install a shared symmetric key k_1 during Provision instead of relying solely on pk_{PVD} . Then, during **I2: Authorization**, PVD first authenticates CSM through standard PKI. Upon successful authentication, CSM generates a fresh symmetric session key k_2 and securely provides it to PVD. PVD then constructs a `Authorize` request encrypted (via authenticated encryption) using k_1 , embedding both the authorization policy and k_2 . After receiving the request, DEV decrypts it using k_1 , recovering k_2 and using it in place of (sk_{CSM}, pk_{CSM}) in subsequent interactions. However, replacing sk_{DEV} with a symmetric key introduces additional trust assumptions. In particular, VRF would need to share the same secret key with DEV in order to validate proofs, requiring VRF to be trusted. This changes the stakeholder model described in Section 4, where $\mathcal{A}_M$ can include VRF. Moreover, by design, a symmetric-key option does not support multiple mutually distrusting VRF-s, since all of them would need access to the same shared secret. One possible workaround is to designate PVD as the sole VRF. Under this model, external parties seeking to validate T_{proof} would query PVD, who performs verification on their behalf.

9 Related Work

Secure DNN Inference with TEEs. To achieve secure inference on edge devices while preserving efficiency, recent work has explored the use of Trusted Execution Environments (TEEs). One approach is to execute the entire inference pipeline inside a TEE while leveraging attestation mechanisms to provide verifiable inference [1, 4, 10, 43]. However, deploying this approach efficiently requires specialized hardware such as custom GPU [43], Intel SGX [10], and NVIDIA H100 [4]. As such, they are unsuitable for the low-cost edge devices targeted in this work.

A more lightweight option is to partition inference between trusted and untrusted environments. Following the terminology of [19], we refer to these approaches collectively as *TEE-shielded DNN partitioning* (TSDP). The core idea of TSDP is to divide a DNN into protected layers executed inside the TEE and unprotected layers executed by software outside the TEE. Since computation outside the TEE may be observed or tampered with, prior work introduces different ways to safeguard them.

One line of work [15, 38, 47] obfuscates exposed layers and their intermediate activations before offloading execution to untrusted software. The TEE subsequently removes the injected noise and verifies computation integrity. Another line of work designates selected layers as private and executes them within the TEE, while the remaining public layers execute outside the trusted boundary. There, multiple techniques have been proposed to determine which

layers should be classified as public or private. Some techniques optimize partitioning for efficiency, e.g., by separating linear and non-linear layers [19] or minimizing transitions between the TEE and untrusted software [17, 26]. Others identify sensitive layers based on privacy leakage risks via membership inference attacks [25] or model stealing attacks [5, 36].

Approaches based on higher-end TEEs, such as Intel SGX, are not applicable to our target setting due to their hardware requirements. Meanwhile, directly porting such designs to TrustZone, as well as existing TrustZone-based solutions share a common limitation: because inference must execute within the Secure World, they require additional inference-related software to reside inside the Secure World. This increases the TCB size, unnecessarily expanding the attack surface. In contrast, VECODI avoids this limitation by executing inference code in SHANGRI-LA with the same privilege level as (but shielded from) untrusted software. Moreover, since VECODI modifies neither the model architecture nor the inference pipeline, it is compatible with many TDSP partitioning strategies. Table 5 qualitatively compares VECODI with prior work based on TrustZone based on goals **G1-G3**.

Verifiable Execution and Isolation on Edge Devices. Our SHANGRI-LA abstraction resembles enclave-based execution models found in higher-end platforms such as Intel SGX. However, ARM TrustZone (both TrustZone-A and TrustZone-M) does not natively provide an enclave abstraction that enables security-sensitive code to execute in the Normal World with elevated privileges (TrustZone-M does not even feature virtual memory to support user-space processes). Prior studies [2, 12] introduce enclave-based execution for ARM TrustZone-A. However, these techniques are not directly applicable to low-end TrustZone-M devices: Penglai [12] relies on a MMU to enforce isolation, while Sanctuary [2] requires the deployed devices to be multi-core. Such hardware features are typically unavailable on low-end edge devices. For the low-end class of devices targeted in this work, several prior efforts have focused on specific security services, including software integrity verification [11, 28, 29, 32], verifiable function execution [27, 30, 34], secure software updates [7, 8], protection of confidential sensor inputs [9, 31], and compartmentalization [42]. While these approaches provide important building blocks, none of them achieves the full set of goals targeted in this work by themselves.

10 Conclusion

Providing strong security guarantees for DNN inference on low-end edge MCUs is challenging due to limited resources and adversarial deployment settings. In this work, we present VECODI, a system architecture for confidential and verifiable DNN inference on ARM TrustZone-M platforms. VECODI builds on a new execution abstraction, SHANGRI-LA, which introduces a third execution environment in the Non-Secure World while enforcing strong isolation via Secure-World controls. This design allows inference code to run without expanding the Secure-World TCB, while still protecting sensitive model components and enabling verifiable execution. Our evaluation shows that VECODI incurs low overhead in TCB size, memory (RAM/Flash) footprint, and latency, making it viable for resource-constrained MCUs.

References

- [1] Sebastian P Bayerl, Tommaso Frassetto, Patrick Jauernig, Korbinian Riedhammer, Ahmad-Reza Sadeghi, Thomas Schneider, Emmanuel Stappf, and Christian Weinert. 2020. Offline model guard: Secure and private ml on mobile devices.
- [2] Ferdinand Brasser, David Gens, Patrick Jauernig, Ahmad-Reza Sadeghi, and Emmanuel Stappf. 2019. SANCTUARY: ARMing TrustZone with User-space Enclaves. In *NDSS*, Vol. 100. 102.
- [3] Adam Caulfield, Norrathep Rattanavipanon, and Ivan De Oliveira Nunes. 2022. ASAP: reconciling asynchronous real-time operations and proofs of execution in simple embedded systems. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 721–726.
- [4] Prach Chantasantitum, Adam Ilyas Caulfield, Vasisht Duddu, Lachlan J Gunn, and N Asokan. 2026. PAL* M: Property Attestation for Large Generative Models. *arXiv preprint arXiv:2601.16199* (2026).
- [5] Miguel Costa, Tiago Gomes, and Sandro Pinto. 2025. SecureQNN: Shielding the Intellectual Property of QNNs in TinyML Systems. *IEEE Internet of Things Journal* 12, 21 (2025), 44642–44655. doi:10.1109/JIOT.2025.3600300
- [6] Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, Ian Nappier, Meghna Natraj, Tiezheng Wang, et al. 2021. Tensorflow lite micro: Embedded machine learning for tinymicrosystems. *Proceedings of machine learning and systems* 3 (2021), 800–811.
- [7] Ivan De Oliveira Nunes, Karim Eldefrawy, Norrathep Rattanavipanon, and Gene Tsudik. 2019. Pure: Using verified remote attestation to obtain proofs of update, reset and erasure in low-end embedded systems. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.
- [8] Ivan De Oliveira Nunes, Sashidhar Jakkamsetti, Youngil Kim, and Gene Tsudik. 2022. Casu: Compromise avoidance via secure update for low-end embedded systems. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [9] Piet De Vaere, Felix Stöger, Adrian Perrig, and Gene Tsudik. 2024. The SA4P framework: Sensing and attestation as a privilege. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. 873–885.
- [10] Vasisht Duddu, Lachlan J Gunn, and N Asokan. 2024. Laminator: Verifiable ML property cards using hardware-assisted attestations. In *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy*. 317–328.
- [11] Karim Eldefrawy, Gene Tsudik, Aurélien Francillon, and Daniele Perito. 2012. Smart: secure and minimal architecture for (establishing dynamic) root of trust. In *Ndss*, Vol. 12. 1–15.
- [12] Erhu Feng, Xu Lu, Dong Du, Bicheng Yang, Xueqiang Jiang, Yubin Xia, Binyu Zang, and Haibo Chen. 2021. Scalable memory protection in the {PENG}LAI enclave. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*. 275–294.
- [13] Soumendu Kumar Ghosh, Arnab Raha, and Vijay Raghunathan. 2023. Energy-efficient approximate edge inference systems. *ACM Transactions on Embedded Computing Systems* 22, 4 (2023), 1–50.
- [14] Javier Gonzalez and Philippe Bonnet. 2014. Tee-based trusted storage. (2014).
- [15] Jiahui Hou, Huiqi Liu, Yunxin Liu, Yu Wang, Peng-Jun Wan, and Xiang-Yang Li. 2021. Model protection: Real-time privacy-preserving inference service for model privacy at the edge. *IEEE Transactions on Dependable and Secure Computing* 19, 6 (2021), 4270–4284.
- [16] Sehyeon Jeong and Hyunyoung Oh. 2024. SeLTZ: Fine-Grained Data Protection for Edge Neural Networks Using Selective TrustZone Execution. *Electronics* 14, 1 (2024), 123.
- [17] Zhaolong Jian, Xu Liu, Qiankun Dong, Longkai Cheng, Xueshuo Xie, and Tao Li. 2025. SmartZone: Runtime Support for Secure and Efficient On-Device Inference on ARM TrustZone. *IEEE Trans. Comput.* 74, 6 (2025), 2144–2158. doi:10.1109/TC.2025.3557971
- [18] Liangzhen Lai, Naveen Suda, and Vikas Chandra. 2018. Cmsis-nn: Efficient neural network kernels for arm cortex-m cpus. *arXiv preprint arXiv:1801.06601* (2018).
- [19] Ding Li, Ziqi Zhang, Mengyu Yao, Yifeng Cai, Yao Guo, and Xiangqun Chen. 2025. Teeslice: Protecting sensitive neural network models in trusted execution environments when attackers have pre-trained models. *ACM Transactions on Software Engineering and Methodology* 34, 6 (2025), 1–49.
- [20] En Li, Liekang Zeng, Zhi Zhou, and Xu Chen. 2019. Edge AI: On-demand accelerating deep neural network inference via edge computing. *IEEE transactions on wireless communications* 19, 1 (2019), 447–457.
- [21] Xiang Liu, Yijun Song, Zelan Li, Jiansheng Chi, Linshan Jiang, and Jialin Li. 2025. Ed-res: Splitting resnet with collaborate distributed inference on edge devices. In *Proceedings of the 2025 ACM CoNEXT Workshop Edge-Cloud Collaboration for AI*. 38–43.
- [22] Ziyu Liu, Yukui Luo, Shijin Duan, Tong Zhou, and Xiaolin Xu. 2023. Mirrornet: A tee-friendly framework for secure on-device dnn inference. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [23] James Ménétrey, Christian Göttel, Anum Khurshid, Marcelo Pasin, Pascal Felber, Valerio Schiavoni, and Shahid Raza. 2022. Attestation mechanisms for trusted execution environments demystified. In *IFIP International Conference on Distributed Applications and Interoperable Systems*. Springer, 95–113.
- [24] Josh Millar, Yushan Huang, Sarab Sethi, Hamed Haddadi, and Anil Madhavapeddy. 2025. Benchmarking ultra-low-power μNPs. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*. 1060–1074.
- [25] Fan Mo, Ali Shahin Shamsabadi, Kleomenis Katevas, Soteris Demetriou, Ilias Leontiadis, Andrea Cavallaro, and Hamed Haddadi. 2020. Darknetz: towards model privacy at the edge using trusted execution environments. In *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services*. 161–174.
- [26] Myungsuk Moon, Minhee Kim, Joonkyo Jung, and Dokyung Song. 2025. ASGARD: Protecting On-Device Deep Neural Networks with Virtualization-Based Trusted Execution Environments. In *NDSS*.
- [27] Antonio Joia Neto, Norrathep Rattanavipanon, and Ivan De Oliveira Nunes. 2025. Pearls: Provable execution in real-time embedded systems. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3765–3782.
- [28] Job Noorman, Pieter Agten, Wilfried Daniels, Raoul Strackx, Anthony Van Herreweghe, Christophe Huygens, Bart Preneel, Ingrid Verbauwhede, and Frank Piessens. 2013. Sancus: Low-cost trustworthy extensible networked devices with a zero-software trusted computing base. In *22nd USENIX Security Symposium (USENIX Security 13)*. 479–498.
- [29] Ivan De Oliveira Nunes, Karim Eldefrawy, Norrathep Rattanavipanon, Michael Steiner, and Gene Tsudik. 2019. {VRASED}: A verified {Hardware/Software} {Co-Design} for remote attestation. In *28th USENIX Security Symposium (USENIX Security 19)*. 1429–1446.
- [30] Ivan De Oliveira Nunes, Karim Eldefrawy, Norrathep Rattanavipanon, and Gene Tsudik. 2020. {APEX}: A verified architecture for proofs of execution on remote devices under full software compromise. In *29th USENIX Security Symposium (USENIX Security 20)*. 771–788.
- [31] Ivan De Oliveira Nunes, Seoyeon Hwang, Sashidhar Jakkamsetti, and Gene Tsudik. 2022. Privacy-from-birth: Protecting sensed data from malicious sensors with VERSA. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2413–2429.
- [32] Ivan De Oliveira Nunes, Sashidhar Jakkamsetti, Norrathep Rattanavipanon, and Gene Tsudik. 2024. Toward remotely verifiable software integrity in resource-constrained iot devices. *IEEE Communications Magazine* 62, 7 (2024), 58–64.
- [33] Sandro Pinto and Nuno Santos. 2019. Demystifying arm trustzone: A comprehensive survey. *ACM computing surveys (CSUR)* 51, 6 (2019), 1–36.
- [34] Norrathep Rattanavipanon and Ivan De Oliveira Nunes. 2025. Slapp: Poisoning prevention in federated learning and differential privacy via stateful proofs of execution. *IEEE Transactions on Information Forensics and Security* (2025).
- [35] STMicroelectronics. 2026. NUCLEO-L552ZE-Q: STM32 Nucleo-144 Development Board with STM32L552ZE MCU. <https://www.st.com/en/evaluation-tools/nucleo-l552ze-q.html>. Accessed: 2026-04-28.
- [36] Zhichuang Sun, Ruimin Sun, Changming Liu, Amrita Roy Chowdhury, Long Lu, and Somesh Jha. 2023. Shadownet: A secure and efficient on-device model inference system for convolutional neural networks. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1596–1612.
- [37] Sasha Targ, Diogo Almeida, and Kevin Lyman. 2016. Resnet in resnet: Generalizing residual architectures. *arXiv preprint arXiv:1603.08029* (2016).
- [38] Florian Tramer and Dan Boneh. 2018. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. *arXiv preprint arXiv:1806.03287* (2018).
- [39] Florian Tramèr, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. 2016. Stealing machine learning models via prediction {APIs}. In *25th USENIX security symposium (USENIX Security 16)*. 601–618.
- [40] Trusted Firmware-M Project. 2026. Corstone SSE-320 with Ethos-U85 Example Subsystem for Ecosystem FVP. <https://tf-m.docs.trustedfirmware.org/en/latest/platform/arm/mps4/corstone320/README.html>. Accessed: 2026-04-28.
- [41] Trusted Firmware Project. 2026. Trusted Firmware-M (TF-M). <https://www.trustedfirmware.org/projects/tf-m/>. Accessed: 2026-04-28.
- [42] Liam Tyler and Ivan De Oliveira Nunes. 2024. Untrusted code compartmentalization for bare metal embedded devices. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 11 (2024), 3419–3430.
- [43] Stavros Volos, Kapil Vaswani, and Rodrigo Bruno. 2018. Graviton: Trusted execution environments on {GPUs}. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 681–696.
- [44] Jinwen Wang, Yuhao Wu, Han Liu, Bo Yuan, Roger Chamberlain, and Ning Zhang. 2023. IP Protection in TinyML. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. doi:10.1109/DAC56929.2023.10247898
- [45] Mingfu Xue, Yushu Zhang, Jian Wang, and Weiqiang Liu. 2021. Intellectual property protection for deep learning models: Taxonomy, methods, attacks, and evaluations. *IEEE Transactions on Artificial Intelligence* 3, 6 (2021), 908–923.
- [46] Amir Yazdanbakhsh, Kiran Seshadri, Berkin Akin, James Laudon, and Ravi Narayanaswami. 2021. An evaluation of edge tpu accelerators for convolutional neural networks. *arXiv preprint arXiv:2102.10423* 1, 6 (2021).
- [47] Han Zhang, Zifan Wang, Mihir Dhamankar, Matt Fredrikson, and Yuvraj Agarwal. 2024. Verisplit: Secure and practical offloading of machine learning inferences across iot devices. *arXiv preprint arXiv:2406.00586* (2024).

Open Science

To support reproducibility and transparency, we provide all source code required to reproduce the experiments and results reported in the evaluation section (Section 7 of this paper. This includes the implementation of VECODI, benchmark scripts, configuration files, and evaluation artifacts used in our experiments. The code repository is available anonymously at: <https://github.com/Anonymous2060/VECODI>. It will be made publicly accessible upon publication/acceptance.